数字生命：一个可测量的生命定义

工程卷（持续更新）

> 何淇 · 与 DeepSeek（AI）协作

> 数字生命系列 · 工程卷（持续更新）

> 日期：2026-11-03

> 本卷为系列合集的分卷版；完整版见系列总合集。各篇单独 DOI 保留，以总合集与各篇发布记录为准。

卷首简介

本卷收录工程部分：门控与稀疏执行（汇总）、冻结子模型的两组最小实验、数字蚊子的最小工程实现，以及工程笔记（持续更新板块）。

和其他卷不同，**工程卷是持续更新的卷**：以后新的工程实验、失败记录、判断修正，都会继续加进"工程笔记"部分，并按新版本重新发布本卷。

目录

第四部分 执行器工程

- 第二十三篇　门控与稀疏执行：一组工程观察（汇总版·待复核）

- 第二十四篇　冻结子模型：接口、协调与分层——两组最小实验（待复核）

- 第二十五篇　冻结模块与可训练大脑：数字蚊子的最小工程实现（待复核）

第五部分 工程笔记

- 第二十六篇　工程笔记 01：门控与模块化的失败、修正与判断（持续更新）

- 第二十七篇　方向观察 01：加层、回放与蒸馏——大模型睡眠巩固（待复核）

第二十三篇 · 门控与稀疏执行：一组工程观察（汇总版·待复核）

> 单独 DOI：https://doi.org/10.5281/zenodo.23858151

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 执行器工程（汇总篇，内部编号第 44 篇）

> 日期：2026-11-01

> 状态：工程观察汇总——单种子为主、数据与算力有限；结论保守，不作方法宣称。

> 说明：本篇将此前分散的执行器工程观察（论文-39、40、41、42 及未发布的 43）合并为一篇。

摘要

执行器工程研究的是一个工程问题：**能不能让模型在推理时"少算"，而质量不明显下降？**

我们的回答是：能，但有明确边界。主要观察如下：

1. **门控训练可用于从零训练的模型**：每层一个内容门 σ(W·h+b)，用拉格朗日乘子把平均开启层数压到目标。在 GPT-2 small（12 层）上，门控模型平均使用 7.48 层，PPL 12.03（plain 对照 16.35，plain 未收敛，不作"门控更优"声称）。

2. **"保留 K/V 的稀疏执行"与稠密门控精确等价**：只算门开 token 的 Q、注意力输出行与 MLP，K/V 全保留；实测 PPL 12.032 vs 12.033（无损）。这证明门控本身不伤质量，质量差异来自"执行方式"。

3. **推理端转 MoD 容量路由，质量几乎无损**：门开优先 + 每层固定容量 K_max + padding，factor=1.0 时名义省 37.9% 计算，PPL 12.107（+0.6%）。

4. **真实加速未取得**：自写 gather/索引实现下，batch 32 墙钟 0.89~1.01x；早期一次实现曾测到 1.22x，说明加速高度依赖实现，非本文可结论。

5. **注意力子集化（跳过 token 不产生 K/V）代价明显**：在已训练模型上 +6.8% PPL；从零训练两次尝试均崩（PPL 350.99 / 393.71）。

6. **规模与训练方式的影响**：从零训练时，1.2M、124M、1.5B 模型都能以约 44%~50% 的层数工作，代价 1%~7%；而预训练模型 + LoRA 后训练（3B/7B）层数压不动、PPL 反升。关键变量是训练方式，不是参数量。

一、问题与背景

1.1 为什么关心"少算"

Transformer 每生成一个 token 都要过完所有层。若某些 token 或某些层对该 token 而言是冗余的，跳过它们就能省算力。这条思路在文献中有多个版本：MoD（Mixture-of-Depths，容量路由）、早退（early exit）、条件计算（conditional computation）、稀疏专家（MoE）等。

我们关心的是其中的"层门控"：**每层、每个 token 一个开/关决策，决定这一层是否对该 token 施加变换。**

1.2 我们的术语（先定义，后文都用这个含义）

- **内容门**：每层一个线性层 W_l，输入该 token 当前的隐藏状态 h，输出 $g = \sigma(W_l \cdot h + b)$。因此门是"每层一个 W_l，但每个 token 每层各有决策"；token 经过每层的矩阵变换后 h 改变，下一层的门分数也随之改变。

- **门开（$p > 0.5$）**：这一层对该 token 计算并写回。

- **门关（$p \leq 0.5$）**：跳过这一层，该 token 的表示保持残差直通。

- **K/V**：注意力的记忆，所有 token 都产生并保留（我们的"保留 K/V"版本）。

- **Q 侧更新**：每个 token 自己的查询与注意力输出；门关时这一部分被跳过。

- **稀疏执行**：按门的决策，只对门开 token 计算 Q/注意力输出/MLP，而 K/V 始终全保留。

- **容量路由（MoD 风格）**：每层固定容量 K，按门分数取 top-K 个 token 进入该层计算（跨层容量不同）。

- **子集化**：更强的一种稀疏——被跳过的 token 不产生 K/V，注意力只在被选中的子集内计算。

1.3 与已有工作的关系

我们不声称提出新方法。门控、稀疏执行、容量路由与早退同属条件计算一类；MoD 已报告过容量路由的可行性。我们的工作是：在有限的算力与数据条件下，把"门控训练 + 推理端稀疏执行"这条工程路线走一遍，记录哪些可行、哪些不可行、边界在哪里。

二、方法

2.1 内容门 + 拉格朗日预算

每层一个内容门。训练时用拉格朗日乘子 λ 控制平均开启比例：损失 = 语言建模损失 + $\lambda \cdot$ 平均活跃层数；λ 按"当前平均活跃 − 目标"自适应调整。这样门不需要人工指定"哪层该关"，由训练自己学。

2.2 两种执行方式

- **保留 K/V 的稀疏执行**：K/V 全算；门开 token 的 Q/注意力输出/MLP 才算。

- **容量路由（MoD 风格）**：每层固定容量 K，按分数选 top-K token；其余 token 走残差。为保证形状静态，用 padding 席位固定 K_max，门开优先填满，不足的空位只算不写回。

2.3 实验设置（统一说明）

- 数据：tinyshakespeare（约 1MB）或 TinyStories 切片（20MB/100MB）。

- 评测：验证集 PPL；以 best 记录（早停，避免过拟合干扰）；单种子。

- 硬件：阿里云 PAI-DSW，单卡 V100 16GB 或 A10 24GB。

三、实验与结果

3.1 小模型（**6** 层，字符级，约 **1.2M** 参数）

早期实验，从头训练：

- plain 3 种子均值 PPL 6.103（6.232 / 5.999 / 6.083）；

- 内容门 + 拉格朗日目标 3 层：PPL 6.028，平均 2.97/6 层（省约一半）；

- 位置门（每层×位置固定表）：目标 3 层 → PPL 6.038，平均 2.98 层；

- λ=0 自由训练时，24 层小模型自己选择 5.3/24 层。

观察：小模型上门控可以用一半层数而不降质量；门是自己学出来的，不是人工指定。

3.2 GPT-2 small（**12** 层，**124M**），从零训练

1MB 莎士比亚：

| 组 | best PPL | 步数 | 平均层数 |

|---|---|---|---|

| plain | 44.03 | 1500 | 12 |

| joint_gate | 47.01 | 1000 | 5.30（44%） |

代价 +6.8%。门控在 λ≈0 时自己收敛到约 5.3 层。

100MB TinyStories：

| 组 | best PPL | 步数 | 平均层数 |

|---|---|---|---|

| plain | 4.892 | 9000 | 12 |

| joint_gate | 5.023 | 9000 | 5.81（48%） |

代价 +2.7%。每层活跃率（final）：`1.00, 0.82, 0.61, 0.01, 0.12, 0.35, 0.28, 0.25, 0.25, 0.45, 0.75, 0.74`。第 0 层全开、第 3 层几乎不用、末两层回到 0.74~0.75。

观察：数据变多（1MB → 100MB）时，层的使用比例几乎不变（56% → 52%），但代价变小（6.8% → 2.7%）。也就是说，数据多让门控"省得起"，没有让门控"省更多"。

3.3 Qwen2.5-1.5B（28 层），从零训练，100MB

| 组 | final PPL | best PPL | 平均层数 |

|---|---|---|---|

| plain | 6.301 | 6.501 @6500 | 28 |

| joint_gate | 6.373 | 6.113 @6500 | 12.32（44%） |

final 口径代价 +1.1%；best 口径门控组反而更低。每层活跃率显示**最后 9 层（L19~L27）几乎全部关闭**。

观察：模型放大到 1.5B，门控依然便宜（44% 层数、+1.1%）。这与"越大的模型越难压缩"的直觉相反，也说明关键变量是"是否从头训练"。

3.4 预训练模型 + LoRA 后训练：压不动

3B（36 层）与 7B（28 层），加载预训练权重，LoRA r=16，数据 1MB，2000 步：

| 设置 | plain best | 门控 best | 用的层 | 代价 |

|---|---|---|---|---|

| Qwen2.5-3B 目标 33 | 12.61 | 14.94 | 32.79/36（91%） | +18% |

| Qwen2.5-7B 目标 26 | 7.01 | 9.71 | 25.70/28（92%） | +38% |

| Qwen2.5-7B 目标 24 | 7.01 | 12.93 | 24.01/28（86%） | +85% |

| Qwen2.5-7B 目标 22 | 7.01 | 12.79 | 22.27/28（80%） | +82% |

7B 目标 26 组后段还发生门控塌缩（PPL 升到 1243，层数掉到 20）。

观察：预训练权重 + 轻量后训练 + 小数据，层数压不动、质量反升。这不构成"大模型不能压缩"的结论，只能说明"这种后训练方式不行"。

3.5 GPT-2 门控模型 **+** 推理端 **MoD**（容量路由）

在 3.2 的 GPT-2 门控模型（7.48/12 层，PPL 12.03）上测试多种推理方式：

| 推理方式 | PPL | 相对 dense | 平均活跃 | 说明 |

|---|---|---|---|---|

| dense gated（参照） | 12.033 | — | — | 全部计算 |

| open_only（只算门开 token，K/V 全留） | **12.032** | **等价** | 0.622 | 动态形状 |

| open_cap（门开优先 + 容量上限） | 12.176 | +1.2% | 0.592 | 动态形状 |

| global top-k（全局 top-k） | 12.182 | +1.2% | 0.624 | 动态形状 |

| static padded（K_max + mask） | 12.046~12.107 | +0.1%~+0.6% | 0.62~0.77 | 静态形状 |

K_max 余量扫描（dense 参照 12.034；K_max = 平均门开数 × factor）：

| factor | PPL | 平均活跃 | 名义省算力 |

|---|---|---|---|

| 1.35 | 12.046 | 0.767 | 23.3% |

| 1.25 | 12.054 | 0.730 | 27.0% |

| 1.15 | 12.051 | 0.691 | 30.9% |

| 1.05 | 12.098 | 0.649 | 35.1% |

| **1.00** | **12.107** | **0.621** | **37.9%** |

| 0.95 | 14.458 | 0.592 | 40.8% |

观察：质量拐点在 factor 1.0~1.05；质量优先的落点是 factor=1.0，名义省 37.9%、PPL +0.6%。

真实墙钟（重要负结果）：

| 实现 | batch | dense | 稀疏 | 加速 |

|---|---|---|---|---|

| 早期 query 端稀疏 | 1 | 18.9 ms | 20.5 ms | 0.93x |

| 同上 | 8 | 19.7 ms | 20.8 ms | 0.95x |

| 同上 | 32 | 43.4 ms | 35.5 ms | 1.22x |

| 静态填充实现 | 32 | 38.9 ms | 41.7~43.4 ms | 0.89~0.93x |

| 静态填充实现 | 64 | 73.9 ms | 73.5 ms | 1.01x |

自写实现是 Python 级 gather + 自定义注意力，远慢于 PyTorch 融合 kernel。早期 1.22x 与后期 0.89x 的差异说明：**墙钟结果高度依赖实现细节，不能作为稳定结论**。

3.6 注意力子集化：失败

(a) 在已训练模型上直接切换子集化语义（跳过 token 不产生 K/V）：

| 容量 | PPL | 相对 dense |

|---|---|---|

| quantized（名义省 37.5%） | 12.855 | +6.8% |

| half（45.8%） | 14.030 | +16.6% |

| quarter（68.75%） | 18.692 | +55% |

(b) 从零训练子集化（两次尝试）：

| 尝试 | plain PPL | 子集化 PPL | 活跃率 |

|---|---|---|---|

| 门 + 阈值式动态子集 | 19.20 | 350.99 | 塌到 8.3% |

| 门当路由器 + 固定容量 top-k + 课程 | 19.03 | 393.71 | 0.496（容量未塌） |

(c) 短微调恢复：400 步、lr 5e-5，PPL 崩到 136（超参不当，不能作为"微调无效"的证据）。

观察：注意力子集化在我们这套门控路由上代价明显；从零训练更差。但这不是"子集化本身不可行"的结论——MoD 原方法用独立路由器 + 容量路由，在更大规模上验证过该语义。

3.7 分支独立门控：失败

每层两个门（注意力门、MLP 门分开），从零训练 12000 步：final PPL 14.484（注意力门 0.61、MLP 门 0.47）；作为对照，联合门控（一个门管两分支）4000 步就到 12.033。分支独立门控即使训练 3 倍步数仍明显更差。

分支稀疏拆分评测（在分支独立门控模型上，用 top-k 强制压低单边）：

| 测试 | PPL |

|---|---|

| baseline | 14.486 |

| MLP 压到 25% | 558.95 |

| 注意力压到 25% | 69.45 |

| 两边都压 25% | 2195.20 |

观察：在这个模型上，"注意力更新"比"MLP 更新"更耐稀疏。结合"保留 K/V 无损、子集化伤质量"的结果，初步图景是：**MLP（权重承载的变换）与 K/V（记忆）是承重部分，Q 侧更新相对可省**。但这只是单模型观察，不能当结论。

四、讨论

4.1 为什么**"**保留 **K/V"**明显更好

[解释] 注意力是跨 token 的信息通路。被跳过的 token 若从注意力中消失，后续 token 少看到一部分上下文，语言建模难以补偿。门控只决定"是否更新该 token 的表示"；保留 K/V 时，更新与否不影响其他 token 可访问的信息。这是我们看到"保留 K/V 无损、子集化 +6.8%"的直接原因。

4.2 为什么从零训练与后训练差别这么大

[解释] 从零训练时，模型从一开始就在"可能跳层"的约束下学习，可以把计算集中到少数层（1.5B 的最后 9 层几乎全关就是证据）。预训练模型没有这个机会：每一层都被预训练"用惯了"，LoRA 低秩更新改不动这个结构。

4.3 名义省算力 **vs** 真实加速

[事实] 我们测到的是名义计算量（层数×token 比例）。真实墙钟取决于 kernel 实现、批大小、内存访问；我们自写的实现在 batch 32/64 上为 0.89~1.22x，不能作为稳定结论。真实加速是独立的工程问题，超出本实验条件。

五、局限

- 单种子为主；数据规模有限（1MB~100MB）；plain 在部分实验中未收敛；

- 从零训练组的绝对 PPL 不代表模型真实能力（欠训练）；

- 仅 PPL，无下游任务；

- 墙钟测量受实现影响大；

- 未与 MoD 原文做同设置对比；

- 未验证 1B 以上从零训练、未验证跨域泛化。

六、结论

1. **门控训练可用**，且门是"每层一个 W_l、每 token 每层一个决策、跟着 token 表示走"的内容门。

2. **"保留 K/V 的稀疏执行"与稠密门控精确等价**——门控本身不伤质量；质量差异来自执行方式。

3. **推理端容量路由（门开优先 + 静态 K_max）可做到名义省 37.9%、PPL +0.6%**；质量拐点在 K_max ≈ 平均门开数。

4. **注意力子集化在我们这套门控路由上代价明显**（+6.8%；从零训练崩），不推广为子集化本身不可行。

5. **关键变量是训练方式而非参数量**：从零训练时 1.2M~1.5B 都能省一半左右；预训练 + LoRA 后训练压不动。

6. **真实加速是独立的工程问题**，本文不作结论。

附：标注

- **[事实]** 第三节全部 PPL、层数、活跃率、墙钟（来自 results JSON）。

- **[解释]** 第四节的分析。

- **[推测]** "从零训练会把计算集中到少数层"——由 1.5B 层活跃率支持，未在更大规模验证。

- **[不声称]** 不声称新方法；不声称真实加速；不声称与 MoD 原文可比。

- **旧篇关系**：本篇合并论文-39、40、41、42 及未发布的 43；旧篇保留 DOI，视为被本篇取代（superseded）。

第二十四篇 · 冻结子模型：接口、协调与分层——两组最小实验（待复核）

> 单独 DOI：https://doi.org/10.5281/zenodo.23859913

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇（内部编号第 45 篇）

> 日期：2026-11-01

> 状态：最小实验观察——单种子、小规模；不作方法宣称。

摘要

本篇回答一个问题：**已经训练好、被冻结的子模型，能不能通过接口组合起来，形成新的结构？**

我们做了两组互补的最小实验：

实验一（并列组合）：两个独立训练的 6 层字符级模型——A 学"正常文本的下一个字符"，B 学"凯撒位移（+1）文本的下一个字符"。把 A、B 冻结，在它们之间加可训练的总控接口，支持四种模式：A 单独、B 单独、A→B（正常输入、位移输出）、B→A（位移输入、正常输出）。

结果：

- 直接首尾拼接（接口为恒等）：PPL 62.84（崩）；

- 加可训练总控：A→B 5.94，B→A 5.67；

- 原生 12 层模型（同任务上界）：5.41；

- A 单独 6.03，B 单独 6.31。

实验二（分层组合）：一个最小趋化环境（一维圈形跑道、食物消耗与再生、能量与死亡）。感知模块 M1（监督训练）、适应模块 M2（监督训练）冻结，决策层 D 用强化学习在环境里训练。

结果：

- 随机策略：回报 -1.34、平均存活 66 步、死亡率 97%；

- 只用原始传感器 + D：回报 -1.40、存活 45 步、死亡率 98%（学不出来）；

- M1 + D：回报 0.494、存活 183 步、死亡率 10%；

- M1 + M2 + D：回报 0.505、存活 184 步、死亡率 9%。

结论（观察，不是定论）：

1. 冻结子模型不能直接拼接，需要可训练的接口/总控；

2. 加入总控后，并列组合可以接近原生模型（5.94/5.67 vs 5.41）；

3. 分层组合中，冻结的下层特征对上层决策是决定性的（M1 把死亡率从 98% 降到 10%）；

4. 适应模块 M2 在本实验中只带来微小提升（0.494 → 0.505），其价值尚未被环境充分"逼出来"。

一、问题与背景

1.1 为什么要"冻结子模型组合"

训练一个模型昂贵；如果能力可以被压缩进小的、可冻结的子模型，然后把它们组合起来，就能：

- 复用已有能力（不用重训）；

- 替换单个模块而不动其他部分；

- 逐步构建更复杂的结构（分层/模块化）。

但冻结也意味着**不能靠继续训练来补偿**：接口必须自己完成协调。这正是本篇要测的。

1.2 与已有工作的关系

我们不声称新方法。相邻工作包括：

- Model stitching：冻结两半网络，用小缝合层连接，用于表示相似性研究；

- 任务向量/功能向量：能力在权重或激活空间中的可迁移表示；

- 冻结专家组合：多个冻结模块 + 可训练接口/路由（如 Bicameral Model 用可训练接口双向耦合两个冻结语言模型）。

我们的实验是这些思想在小规模上的最小化验证：**接口是否必要、接口能补回多少、分层冻结是否可行。**

1.3 术语

- **冻结子模型**：训练完成后参数不再更新的模型。

- **总控/接口**：插在两个冻结子模型之间的可训练小模块，负责表示空间对齐（本实验中是两个 2 层 MLP）。

- **并列组合**：两个子模型能力不同、通过接口互相衔接（实验一）。

- **分层组合**：下层冻结模块提供特征，上层可训练模块做决策（实验二）。

二、实验一：并列组合（两个冻结语言模型 + 总控）

2.1 设置

- 模型：两个 tiny GPT-2 风格字符级模型（各 6 层、隐藏维度 256、8 头、字符级词表 65）。

- 数据：tinyshakespeare 字符级，序列长度 128。

- 任务：

- A：正常文本的下一个字符；

- B：凯撒位移（+1）文本的下一个字符。

- 训练：A、B 各自从零训练 1200 步（batch 16，lr 3e-4），训练后冻结。

- 总控：每个方向一个 2 层 MLP（256→256→256，GELU），初始近似恒等；训练时只更新总控，A、B 冻结。

- 对照：

- 恒等接口（不训练）；

- 原生 12 层模型（同任务，1500 步），作为"上界"；

- A 单独、B 单独的 PPL。

2.2 模式定义

```

A-only：输入 → A 的嵌入 → A 的 6 层 → A 的 ln_f + lm_head
```

B-only：输入 → B 的嵌入 → B 的 6 层 → B 的 ln_f + lm_head

A→B： 输入 → A 的嵌入 → A 的 6 层 → C_AB → B 的 6 层 → B 的 ln_f + lm_head

B→A： 输入 → B 的嵌入 → B 的 6 层 → C_BA → A 的 6 层 → A 的 ln_f + lm_head

```

**2.3** 结果

| 设置 | 验证 PPL |

|---|---|

| A 单独（正常文本） | 6.03 |

| B 单独（位移文本） | 6.31 |

| 原生 12 层（翻译任务上界） | 5.41 |

| A→B 直接拼（恒等接口） | **62.84** |

| A→B + 训练的总控 | **5.94** |

| B→A + 训练的总控 | **5.67** |

**2.4** 观察

1. **恒等接口完全不行**（62.84）：两个独立训练模型的隐藏表示空间不同，直接首尾相接等于把一种"语言"喂给只懂另一种"语言"的层。

2. **可训练总控能补回来**：A→B 5.94、B→A 5.67，接近原生 12 层上界 5.41，也接近两个子模型各自的水平（6.0~6.3）。

3. **总控非常小**：两个方向共约 26 万参数，相对两个子模型（各约 500 万参数）约 2.6%。

4. **方向基本对称**：A→B 与 B→A 差距很小（5.94 vs 5.67），本实验中未出现明显不对称。

**2.5** 局限

- 字符级 tiny 模型、凯撒位移是极简单任务；

- 原生 12 层上界只训练 1500 步，可能未完全收敛；
```

- 模式是离散切换（四次独立模式），不是"同时双向耦合"；

- 单种子。

三、实验二：分层组合（趋化环境中的冻结感知/适应 + 可训练决策）

3.1 环境

最小趋化环境（纯函数模拟器，CPU 上运行）：

- 一维圈形跑道，64 格；

- 食物场 f[i] ∈ [0,1]：初始 2~3 个高斯热点；被吃后减少；每步按 0.0015 再生；

- 智能体：位置 p、能量 e（初始 0.5，上限 0.8）；

- 每步消耗 0.035 能量；

- 吃当前格食物：最多 0.15×0.6 的补充；

- e ≤ 0 → 死亡，回合结束；

- 观测：前后各 2 格平均食物量（加高斯噪声 0.02）；

- 动作：左移 / 右移 / 停留；

- 每回合最多 200 步；

- 奖励：能量变化 + 0.002（存活）；死亡额外 -1。

3.2 模块

```

M1 感知（监督训练，冻结）：

· 输入：左右传感器读数（2 维）

· 输出：哪边食物多（2 类）

· 标签：模拟器中的真实左右食物量比较（精确可算）

· 训练：4000 个随机位置样本
```

M2 适应（监督训练，冻结）：

· 输入：最近 6 步的传感器均值（6 维）

· 输出：背景食物水平（标量）

· 标签：模拟器中的全场平均食物量

· 训练：3000 个序列

D 决策（可训练）：

· 输入：原始传感器（2）+ M1 输出（2）+ M2 输出（1）

· 输出：3 个动作的 logits

· 训练：强化学习（REINFORCE + 回报归一化），2500 回合

· M1、M2 冻结，只更新 D

```

**3.3** 结果

| 策略 | 回报 | 平均存活步数 | 死亡率 |

|---|---|---|---|

| 随机策略 | -1.34 | 66 | 0.97 |

| 只用原始传感器 + D | -1.40 | 45 | 0.98 |

| M1 + D | 0.494 | 183 | 0.10 |

| M1 + M2 + D | 0.505 | 184 | 0.09 |

**3.4** 观察

1. **环境有真实压力**：随机策略 97% 会饿死，平均只活 66 步。

2. **只有原始传感器时决策层学不出来**：回报 -1.40、死亡率 98%，与随机策略相当。单步带噪观测不足以让一个小 MLP 用策略梯度学到趋化。
```

3. **冻结的感知模块是决定性的**：加入 M1 后，回报从负转正（0.494）、存活步数 183、死亡率降到 10%。

4. **适应模块 M2 只带来微小提升**：0.494 → 0.505。可能原因：本环境背景变化缓慢，M2 提供的信息增量有限；也可能 M2 的监督目标（全场均值）与决策需要不完全一致。

5. **工程上可行**：全部为小 MLP + CPU，几分钟完成训练与评测。

3.5 局限

- 一维、离散格、简化食物动态；

- M1 与 M2 的功能在一维设定下有重叠，二者的独立贡献未被完全分离；

- RL 只训练 2500 回合，可能未收敛；

- 单种子。

四、讨论

4.1 为什么恒等接口会失败

[解释] 两个独立训练模型各自形成了自己的隐藏表示坐标（尺度、方向、特征基）。A 的最后一层输出是"给 A 的输出层用"的表示，不是 B 的输入层期望的表示。直接相接等于跨语言通信而不做翻译，失败是预期的。

4.2 为什么小总控能补回来

[解释] 总控只需要做"表示空间之间的映射"，不需要重新学习任务。映射是一个低维线性/非线性变换，参数量远小于子模型，但足以把 A 的表示搬到 B 的输入分布附近。实验中 2.6% 的参数补回了 ~90% 的差距（62.84 → 5.94，上界 5.41）。

4.3 为什么分层组合中下层冻结是决定性的

[解释] 决策层只看到观测与下层特征。如果观测本身信息不足（单步带噪传感器），策略梯度很难从稀疏奖励里学到有效行为；而冻结的感知模块把"哪边食物多"这个判别特征直接提供给决策层，等价于给了一个好的状态表示。这与"预训练特征 + 线性探测/微调头"的常见经验一致。

4.4 关于"感知与适应是否重复"

[事实] 一维设定下 M1 与 M2 的功能确有重叠（都在处理传感器历史/差异）。本实验没有分离出"历史信息"与"感知特征"各自的贡献。要回答这个问题，需要加入"raw_hist"（原始历史但无 M1/M2）与"循环决策层"两组对照。

五、局限（总）

- 两个实验均为单种子、极小规模（字符级 tiny 模型 / 一维环境）；

- 实验一的"上界"模型训练步数少于子模型，比较不严格；

- 实验二的 RL 训练量有限，M2 的收益可能被低估；

- 未做"同时双向耦合"（lockstep）与"逐层冻结生长"的实验；

- 未验证真实规模模型上的可迁移性。

六、结论

1. **冻结子模型不能直接拼接**；需要一个可训练的接口/总控来完成表示对齐（实验一：62.84 → 5.94）。

2. **小总控（约 2.6% 参数）可以补回大部分损失**，使并列组合接近原生模型（5.94/5.67 vs 5.41）。

3. **分层组合成立**：冻结的感知模块 + 可训练决策层，在趋化环境中把死亡率从 98% 降到 10%，回报由负转正。

4. **适应模块的价值未显现**（0.494 → 0.505），需要更"需要适应"的环境来检验。

5. 工程实现无特殊门槛：小 MLP + CPU 即可完成，接口与模块都可以冻结/替换。

附：标注

- **[事实]** 第二、三节全部 PPL、回报、存活步数、死亡率（来自运行结果）。

- **[解释]** 第四节的分析。

- **[推测]** "适应模块在更难环境中会有价值"——未验证。

- **[不声称]** 不声称新方法；不声称优于原生模型；不声称可扩展到真实规模。

- 相邻工作：model stitching、任务/功能向量、冻结专家组合（如 Bicameral Model）。

第二十五篇 · 冻结模块与可训练大脑：数字蚊子的最小工程实现（待复核）

> 单独 DOI：https://doi.org/10.5281/zenodo.23890632

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 执行器工程（内部编号第 46 篇）

> 日期：2026-11-03

> 状态：最小工程实现与观察——单种子、小规模；不作方法宣称。

> 说明：本篇与论文-45《冻结子模型：接口、协调与分层》是同一原理的两种形态：论文-45 做的是两个冻结语言模型之间的拼接与协调；本篇做的是"冻结的感知/执行模块 + 可训练决策大脑"。

摘要

本篇回答一个工程问题：**如果感知和执行都交给冻结模块，只让"大脑"可训练，一个小系统能学会生存行为吗？**

我们做了一个"数字蚊子"的最小实现：

- **冻结的眼睛**：12 条 360° 射线，返回归一化距离（固定算法）；

- **冻结的翅膀**：转向动力学 + 速度一阶低通（固定算法）；

- **可训练的大脑**：64×64 两层 MLP，输出（转向，推力）；

- **环境**：二维场地，障碍物定期从边缘生成、朝蚊子当前位置飞来，碰到即死；

- **训练**：进化策略（ES）——本质就是"存活→参数保留→下一代"的演化。

主结果：

| 配置 | 平均存活步数 | 中位存活 | 被撞率 |

|---|---|---|---|

| 完整（冻结眼 + 冻结翅 + 演化脑） | **316.9** | 400（跑满） | 42.7% |

| 无眼睛（只有速度，盲飞） | 68.0 | 66 | 100% |

结论（观察，不是定论）：

1. 冻结的感知模块是决定性的：有眼睛能学会左右闪躲、平均活 317 步；没有眼睛几乎必死（68 步、100% 被撞）。

2. 本篇用固定算法作为"冻结模块"的最简形式；对上层可训练部分来说，它与"训练后冻结的网络"同样是"不可改、只能使用"的映射（一般情形的对照是后续工作）。

3. 演化策略（ES）在这类小系统上比策略梯度更稳定：早期的 REINFORCE 版本会学出"保守不动"或"自杀"策略。

4. 任务设计比算法更关键：我们先做"追目标 + 避障"，学会的是"躺平"或"冲锋"；换成"障碍冲向我、只能左右躲"后，行为才干净地出现。

一、问题与背景

1.1 冻结模块能不能组合、复用

论文-45 已经观察过：两个独立训练的语言模型冻结后直接拼接会崩（PPL 62.8），加一个可训练的小总控能恢复（5.94）；分层组合中，冻结的感知模块 + 可训练决策层能在趋化环境里把死亡率从 98% 降到 10%。

那组实验里的"冻结模块"都是训练好的神经网络。本篇的自然延续问题是：**冻结的（有参数的）模块 + 一个很小的可训练大脑，能不能在更接近"感知—动作"的闭环里学会生存行为？**

为了把问题压到最小，本篇的冻结模块用固定算法实现（射线检测与低通动力学）。它们是"冻结模块"的最简形式：参数（这里是算法常数）固定、不参与训练。更一般的情形——训练后冻结的神经网络——是后续工作；本篇不主张"固定算法"与"训练后冻结"在所有方面等价，只主张：对上层可训练部分来说，两者都是"不可改、只能使用"的映射。

1.2 生物原型：蚊子

蚊子是很好的原型：神经元少、视觉发达、时间尺度快（翅膀每秒几百次、反应几十毫秒）。它没有时间"想"每一次拍翅——快速回路必须是硬连线的（冻结），只有"往哪飞、躲谁"这类慢决策交给可学习/可演化的部分。

所以"数字蚊子"的结构很自然：

```
眼睛（冻结）→ 大脑（可训练/可演化）→ 翅膀（冻结）
```

1.3 两种"冻结"的来源

本篇使用"冻结模块"这个说法时，包含两种来源：

- **先天规则型**：固定算法（射线检测、低通动力学），参数固定、不参与训练；

- **训练后冻结型**：先训练一个小网络，再把参数冻结。

对上层可训练部分来说，这两种模块是等价的：都是"参数不再更新、只能使用"的映射。

二、实验设计

2.1 环境

- 二维场地 $[-1, 1]^2$；

- 蚊子：位置、朝向、速度；出生在中心附近；

- 障碍物：每 12 步从边缘生成一个，半径 0.025–0.055，朝蚊子当前位置飞行，速度 0.018–0.03/步；同屏最多 10 个；

- 碰撞（距离 < 障碍半径 + 0.02）→ 死亡，回合结束；

- 每回合最多 400 步，活满算"存活"。

2.2 冻结模块

眼睛（冻结）：

- 12 条 360° 射线，对每个障碍做射线-圆求交，取最近距离；

- 输出 12 维归一化距离（除以最大量程 0.8），再拼接当前速度（除以最大速度），共 13 维；

翅膀（冻结）：

- 转向：朝向 += 转向指令 × 0.35 弧度/步；

- 速度：一阶低通，speed += 0.25 × (目标速度 − speed)，最大速度 0.05/步；

2.3 可训练的大脑

- 输入：13 维（眼睛输出 + 速度）；

- 结构：Linear(13, 64) → Tanh → Linear(64, 64) → Tanh → Linear(64, 2) → Tanh；

- 输出：转向（[-1, 1]）、推力（[0, 1]）。

2.4 训练：进化策略（**ES**）

- 群体 32；每代对每个个体加高斯噪声（σ=0.06），确定性评估 2 个回合；

- 适应度 = 平均回报 + 2 × 平均存活步数；

- 按排名加权更新参数（OpenAI-ES 风格），学习率 0.05，共 180 代。

选 ES 而不是策略梯度的原因见 4.3：REINFORCE 版本不稳定，会收敛到"保守不动"或"自杀"。

2.5 奖励

- 每步存活：+0.02；

- 距离惩罚：12 条射线中，距离归一化值低于 0.35 的，按"越近罚越重"累加（权重 8×超出量）；

- 碰撞：−100，回合结束；

- 活满 400 步：+0.5。

三、结果

3.1 主结果

| 配置 | 平均存活步数 | 中位存活 | 被撞率 |

|---|---|---|---|

| 完整（冻结眼 + 冻结翅 + 演化脑） | **316.9** | 400（跑满） | 42.7% |

| 无眼睛（只有速度） | 68.0 | 66 | 100% |

（每种配置评估 300 个回合；单种子训练。）

3.2 观察

1. **冻结的眼睛是决定性的**：有眼睛的个体学会了左右闪躲，平均活 317 步，接近一半回合能撑满 400 步上限；没有眼睛的个体只会盲飞，平均 68 步、100% 被撞。

2. **小脑可以很小**：可训练部分只有 13→64→64→2 的两层 MLP，约 5 千参数；学习只发生在这一小块里。

3. **冻结模块确实"可用但不可改"**：大脑只能学怎么读眼睛、怎么驱动翅膀，不能改射线或动力学；这和"先天回路 + 可塑决策"的结构一致。

3.3 一个反例（先做的版本）

在换成"躲避"任务之前，我们做过"追目标 + 避障"的版本：

| 配置 | 到达率 | 撞障碍 | 撞墙 | 超时 |

|---|---|---|---|---|

| 完整（有眼睛） | 33% | 8.7% | 7% | 51% |

| 无眼睛 | 80.7% | 19.3% | 0% | 0% |

有眼睛反而更差：它看到障碍后变得保守，大量回合超时；没有眼睛的个体径直冲向目标，反而到达率高。

这个反例说明：**任务设计比算法更关键。** 在"追目标"任务里，眼睛提供了"危险的信号"，但没有提供"绕过去"的明确梯度；在"躲避"任务里，生存本身就是唯一目标，眼睛的信息才直接变成行为。

四、讨论

4.1 "冻结"的两种来源是等价的

本篇的眼睛和翅膀是固定算法（先天规则型），不是训练后冻结的网络。但对上层大脑来说，它们与"训练后冻结的网络"没有区别：参数不更新、行为固定、只能被使用。所以论文-45 的结论（冻结模块 + 可训练接口可以组合）在这里以另一种形态成立：**冻结组件 + 可训练的上层适配**。

4.2 演化与学习

ES 的更新规则（采样→按适应度加权→保留）本身就是"存活→参数保留→下一代"的演化模拟。这与之前的 RL 实验不同：RL 是个体在一生中学习；ES 是群体跨代演化。生物两者都有：先天回路（冻结）、一生学习（可塑）、代际遗传（演化）。

4.3 为什么用 **ES**

早期我们用 REINFORCE 训练策略，遇到两个问题：

- 单步回合时回报方差为 0，归一化产生 NaN；

- 更重要的是，策略梯度会学到两类退化解：

- **保守不动**：不动就不会靠近障碍、不会被罚，但也不前进；

- **自杀**：快速结束回合，避免持续的距离惩罚。

ES 用确定性评估和群体选择回避了这两个问题；再加上"死亡 −100 远大于累计惩罚"的设计，退化解被排除。

4.4 异构模块的换装

"冻结模块可以拼接"成立，但有条件：

- 同构/同分布：几乎直接可用（论文-45 中同维度 A、B 只需一个小总控）；

- 异构（换了传感器、执行器或表示）：上层需要重新适配，或者加一个对齐层再训练。

本篇的眼睛与翅膀是同一个人设计的固定算法，接口是"大脑的输入层"——它学习怎么读眼睛特征。如果换成训练后冻结的视觉网络，大脑需要重新训练或加对齐层。

五、局限

- 单种子训练；ES 本身有评估噪声（训练中无眼睛组的适应度曾到 800，但最终评测只有 68 步，说明有波动/过拟合训练随机种子）；

- 环境高度简化（二维、圆障碍、固定生成规则）；

- 冻结模块是固定算法，不是训练后冻结的网络；两种来源在本篇中只是概念上等价，没有做对照实验；

- 没有做多代繁殖/交叉、环境变化、多种子统计；

- 指标只有存活步数和被撞率，没有轨迹质量、能耗等。

六、结论

1. **冻结模块 + 可训练小脑**可以在最小规模上学会生存行为：有眼睛时平均存活 317 步，盲飞只有 68 步。

2. **冻结模块可以是固定算法，不必是训练过的网络**；对上层来说，"冻结"的含义是"不可改、只能使用"。

3. **任务设计决定行为质量**：同样的结构，在"追目标"任务里学会躺平或冲锋，在"躲避"任务里才学会左右闪躲。

4. **ES 比 REINFORCE 更稳**：群体选择天然过滤退化解；ES 也是"演化"这一层的直接模拟。

5. 与论文-45 的关系：两者是同一原理的两种形态——冻结组件 + 可训练的适配/决策层；拼什么、怎么拼，取决于异构程度和接口。

附：标注

- **[事实]** 第三节全部存活步数、被撞率、到达率（来自运行结果 JSON）。

- **[解释]** 第四节的机制分析。

- **[推测]** "如果换成训练后冻结的视觉网络，大脑需要重训"——未做对照实验。

- **[不声称]** 不声称这是数字生命的实现；不声称方法新颖；不声称结果可扩展到真实规模。

- 相邻工作：论文-45（冻结子模型拼接）、model stitching、进化策略（OpenAI-ES）。

第二十六篇 · 工程笔记 **01**：门控与模块化的失败、修正与判断（持续更新）

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 工程笔记（新板块第 01 篇）

> 日期：2026-11-03

> 定位：过程记录，不是结果论文。写我们怎么想、哪里错了、修正成什么、还剩什么没搞清。以后按 02、03…… 继续加。

一、这篇笔记是什么

正式论文只写"我们做成了什么"，工程笔记写"我们是怎么走到那一步的"。

它记录三类东西：

- **最初的设想**：我们以为门是什么、想用它做什么；

- **失败的实验**：哪些方向试过、怎么失败、失败给出什么信息；

- **形成的判断**：从失败和对照里沉淀下来的经验，以及哪些判断还只是初步的。

写它的原因很简单：失败里包含的信息，往往比成功的数字更有用；而且如果不写下来，过几个月就只剩结论，过程没了。

二、最初的设想

最早我们想的门不是"省算力"，而是**判定**：

```

· 在框架里，时间是算子序列：s_{t+1} = I(s_t)

· 判定 J 决定：这一步做什么、什么时候停

· 门控 = 判定的工程实现：

· 门开 → 这一步算

· 门关 → 这一步跳过

⇒ 门是"给每个 token 分配步数/时间"的东西

```

所以第一版的门很小：每层一个 `Linear(d, 1)`，`g = sigmoid(W·h)`，乘在该层的残差贡献上。它不决定输出什么字，只决定这一层对这个 token 算多少。

三、失败的实验

① 注意力子集化（跳过 token 不产生 K/V）

在已训练的门控模型上直接切换语义：PPL 12.033 → 12.855（+6.8%）。从零训练两次尝试都崩：350.99（活跃率塌到 8.3%）、393.71（容量没塌也崩）。

结论：在我们的门控路由上，"被跳过的 token 从注意力里消失"代价明显。但不推广为"子集化本身不可行"——MoD 原方法用独立路由器 + 容量路由，在更大规模验证过。

② 分支独立门控（注意力门、MLP 门分开）

从零训练 12000 步：PPL 14.484；联合门控 4000 步就到 12.033。分支拆分不是更好，而是更差。分支拆分评测还显示：MLP 压到 25% → PPL 558.95；注意力压到 25% → 69.45。

结论：MLP（权重承载的变换）与 K/V（记忆）是承重部分，Q 侧更新相对可省。

③ 预训练 3B/7B + LoRA 后训练

层压不动（80%~92%），PPL 反升（+18%~+85%），7B 还出现门控塌缩（PPL 1243）。

结论：这条只说明"预训练 + 轻量后训练 + 小数据"不行，不说明"大模型不能压"。要回答后者需要从零训练大模型，成本是主要障碍。

④ 追目标任务里"有眼睛反而不如盲飞"

同一套冻结眼 + 冻结翅膀 + 可训练大脑：

| 配置 | 到达率 | 撞障碍 | 撞墙 | 超时 |

|---|---|---|---|---|

| 有眼睛 | 33% | 8.7% | 7% | 51% |

| 无眼睛 | 80.7% | 19.3% | 0% | 0% |

有眼睛的个体看到障碍后变保守，大量超时；没眼睛的径直冲目标，反而到达率高。

结论：任务设计比算法更关键。眼睛提供"危险信号"，但不提供"绕过去"的梯度；换成"躲避"任务后，眼睛的信息才直接变成行为。

⑤ 近距离惩罚让策略学会"躺平/自杀"

在"追目标 + 避障"任务里加"离障碍越近罚越重"后：完整版到达率 0%、超时 99.7%（学会了原地不动）；另一版学会快速结束回合以躲避持续惩罚。

结论：惩罚设计要防止退化解；死亡代价必须远大于累计惩罚，且不能给"停下来"任何好处。

四、修正

从上面这些失败里，我们修正了几件事：

```

· 保留 K/V 的稀疏执行与稠密门控精确等价（12.032 vs 12.033）

⇒ 门控本身不伤质量；质量差异来自执行方式

· 软门是本体，硬跳是省算力的"硬化"

⇒ 训练用软门，部署再硬化

· 判断"能不能压"的关键变量是训练方式，不是参数量

⇒ 从零训练 1.2M~1.5B 都能用约一半层；预训练+LoRA 压不动
```

· 冻结模块 + 可训练接口是同一原理的多种形态

⇒ 论文-45 的拼接、论文-46 的数字蚊子都在这条线上

· 任务设计优先于算法选择

⇒ 同样的结构，换个任务就从"躺平"变成"会躲"

```

## 四之补、约束方式的差别：从 **LAMBDA** 到拉格朗日目标

还有一个容易被忽略的差别：**同一个门，被不同的约束训练，行为完全不同。**

最早那版（论文-38 风格）用的是一项目标正则：

```

loss = 语言建模损失 + LAMBDA × 门权重的平均值

```

它只是"鼓励门变小"，没有"平均只能开几层"的目标。结果是：门大量保持开着（开着能降损失，那点正则压不住），主要的收益是适配（PPL 大幅下降），稀疏性没有保证。

后面从零训练时换成了拉格朗日约束：

```

· λ 自适应调整

· 目标：平均开层数 = k（比如 6 层）

· 不达标就加大 λ，超标就减小

⇒ 门必须在"降损失"和"少开层"之间取舍

```

差别总结：

| | 最早的 LAMBDA | 后面的拉格朗日目标 |

|---|---|---|
```

| 约束对象 | 门权重大小 | 平均开层数 |

| 是否可控 | 不可控（λ 固定，效果不确定） | 可控（按目标调） |

| 目的 | 让门别太大 | 真的压层数 |

| 结果 | 主要适配，稀疏性弱 | 稀疏、可测、可扫曲线 |

所以"门学到了什么"取决于被什么约束：没有目标约束时，它是软适配器；有目标约束时，它才是可调预算的稀疏开关。

五、今天的对照：**sigmoid** 门 **vs** 纯线性门

为了确认"门要不要压缩（sigmoid）"，我们做了一组最小对照：

```

· 冻结 GPT-2 small（不更新）

· 只训练 12 个门（每层 Linear(768→1)，约 9k 参数）

· 数据：莎士比亚 1MB，训练 600 步

```

结果：

| 组 | 验证 PPL |

|---|---|

| 无门（冻结基线） | 44.31 |

| **sigmoid 门** | **24.59** |

| 纯线性门（无 sigmoid） | 27.48 |

门值分布：

```

· sigmoid 门：全部在 0~1，层间分化明显，稳定

· 纯线性门：出现负值（最低 -27.3）和极大值（最高 17.5）
```

· 层 11 方差 2.5，训练不稳定

⇒ 无界放大/取负没有带来收益，反而更差

```

结论：门需要压缩到有界区间（sigmoid）；纯线性门不是"更自由"，而是"更不稳定"。

## 六、形成的判断（当前版本）

```

① 门是"每层一个、按 token 内容算开度"的开关

· 不是每 token 一个门参数；参数按层共享，开度按 token 算

② 门的三重身份：

· 时间/步数分配器（最初动机）

· 稀疏开关（省算力）

· 轻量适配器（冻结模型 + 9k 参数，PPL 44→25）

③ 软混合是本体，硬跳是硬化；不考虑算力时应停在软门

④ K/V 必须保留；MLP 是承重墙

⑤ 关键变量是训练方式，不是参数量

⑥ 数据越多，"能压的层"可能越少（初步观察，未定）

⑦ 冻结模块 + 可训练接口可以组合，但接口要对齐

⑧ 任务设计比算法选择更决定行为质量

```

## 七、还没搞清

```

· 数据量曲线：压缩比是收敛到渐近线，还是持续变难？

· 迭代收敛：冻结前端 + 可训练循环块，能否用迭代换精度？

· 模块跨代演化：先天回路本身怎么"演化出来"？

· 门作为解释：门模式能否稳定地解释"模型为什么这样答"？

· 异构模块接口：换传感器/执行器后，上层要重训多少？

· 大模型从零 + 门控：3B/7B 从零训练是否能压？成本是障碍

```

## 八、更新方式

以后每有什么值得记的过程、失败对照、判断修正，就在这里加一篇：

```

· 工程笔记 02：……

· 工程笔记 03：……

```

不追完整，只追"当时怎么想、后来怎么改"。这样这条工程线的过程不会丢。

> 一句话：正式论文写"结果"，工程笔记写"我们怎么从错的地方走到对的地方"。

# 第二十七篇 · 方向观察 **01**：加层、回放与蒸馏——大模型睡眠巩固（待复核）

> 单独 DOI：https://doi.org/10.5281/zenodo.23927099

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 执行器工程 / 方向观察（内部编号第 47 篇）

> 日期：2026-11-04

> 状态：方向观察与最小实验——极小模型、字符级、单种子；只验证机制，不作方法宣称。

## 摘要
```

一个训练完成的模型，如何在不遗忘旧能力的前提下继续学新东西？本篇记录一组最小实验，并给出两个应用方向的判断。

基础是一个 1.21M 参数的字符级 6 层模型：day1 数据为莎士比亚，day2 为 Python 源码。主要观察：

1. **加层能学新能力，且比全参微调少遗忘**：全参微调 day2 时 day1 PPL 从 4.64 崩到 46.3；冻结基础加 6 层后 day1 为 18.2、day2 为 4.26；加层并回放 day1 后 day1 保到 4.83、day2 为 6.60。

2. **门有三个作用**：轻量适配（冻结模型只训门，PPL 44.31→24.59）、步骤分配（每 token 每层决定算不算）、模式切换（单模式回放时零和；双模式回放可两边都保）。

3. **极小回放可以接近全量回放**：100 条旧样本、10% 混合，day1 9.05、day2 5.02（对照：不回放 day1 18.2；全量回放 day1 4.78、day2 8.67）。

4. **蒸馏压缩可以打包**：教师（12 层 gated，已巩固）day1 6.96/day2 4.50，学生（8 层更宽、无门）day1 7.42/day2 2.18。

据此给出两个方向（都只到"可行"级别）：个人 AI 的睡眠巩固；大模型新语料的增量更新。并明确：实验是玩具规模，不声称优于现有做法。

一、问题

模型训练完、部署后，知识停在训练那一刻。要更新，有几条路：

- **外部记忆**：检索、上下文。便宜、无损，但不改变模型本身；

- **全参微调/继续训练**：便宜一些，但会灾难性遗忘（我们实测 day1 PPL 从 4.64 到 46.3）；

- **完全重训**：质量最好，但要全部旧数据 + 新数据，成本最高；

- **冻结旧模型 + 加层**：旧模型只做推理，新层学新东西。

本篇关心最后一条：加层能否承载新能力、如何防止干扰旧能力、如何把生长打包。

二、预备：模型、加层与门

2.1 基础模型

```

· 字符级语言模型：6 层 Transformer
```

· 宽度 d=128，4 个注意力头，MLP 隐层 512

· 参数量 1.21M

· 在 day1（莎士比亚）上从零训练到收敛

· 训练后冻结：参数不再更新

```

用这么小的模型，是为了让所有实验在 CPU 上几分钟跑完，把机制问题与规模问题分开看。

**2.2"**加层**"**是什么

```

· 基础模型 6 层，冻结

· 在它上面再接 6 层新的 Transformer

· 新层随机初始化

· 输入输出维度相同（128）

· 共享冻结的嵌入层、位置编码、最终 LayerNorm 和输出头

⇒ 总模型 12 层：

前 6 层 = 冻结的旧模型

后 6 层 = 新加的可训练层

```

**2.3"**门**"**是什么

门是每层一个很小的模块，决定这一层对当前 token 起多大作用：

```

· 每层一个线性层：Linear(d → 1)，d=128

· p = sigmoid(W·h + b)，一个 0~1 的数（h 是该层输入）

· 这一层前向变成：

h_out = h + p · (Block(h) − h)

· p≈1：这一层完整计算

· p≈0：这一层近似跳过

· 中间：按比例混合

⇒ 门不产生输出，只控制"这一层算多少"

```

门的位置逐实验说明：

- **后 6 层门**：只给新加 6 层加门，前 6 层不加（等价 p≡1）；

- **12 层全加门**：前 6 层（冻结权重）和后 6 层都加门，前 6 层的门也可训练。

门初始化：使开率起始约 0.5~0.9（接近开）。

**2.4** 训练流程

```

阶段 1（day1）：训练基础 6 层（无门）

阶段 2（day2）：冻结基础，只训新 6 层（可带门）

阶段 3（巩固）：混合 day1/day2 训练（可只训门）

阶段 4（蒸馏）：用混合数据训练一个稠密学生

```

## 三、门的三个作用

**3.1** 轻量适配

```

· 冻结模型，只训练门（论文-38：12 个门约 9k 参数）

· GPT-2 small 冻结 + 12 个门：PPL 103.1 → 64.1

· 我们的小实验：44.31 → 24.59

⇒ 门像一个极小的适配器，把冻结模型的层贡献重新加权

· 但它不能学新表示，只能重新组合已有层

```

**3.2** 步骤分配

```

· 每个 token、每一层：算还是不算

· 从零训练时门开率会学出结构：

· 第 0 层几乎全开

· 中高层对部分 token 关闭

· 我们 1.5B 从零训练的模型：最后 9 层几乎全关

⇒ 门在分配"这个 token 走几步"

```

**3.3** 模式切换

```

· 不同数据/模式之间切换层的开合

· 单模式回放：零和

· 只回放 day1：day1 恢复 4.72，day2 崩到 1415

· 双模式回放：两边都保

· day1 6.96（从 26.70 恢复），day2 4.50

⇒ 门能做切换，但要"两种模式都见过"才不零和

```

## 四、最小实验

### **4.1** 单任务上加 **6** 层

在 day1 上训练 6 层基础模型，然后加 6 层：

| 组 | 配置 | 验证 PPL |

|---|---|---|

| A | 基础 6 层 | 4.34 |

| B | 冻结 6 + 后 6 层可训练（门只在后 6 层） | 4.16 |

| C | 冻结 6 + 后 6 层可训练（12 层全加门） | 4.17 |

| D | 冻结 6 + 后 6 层可训练（不加门） | 4.20 |

观察：加层带来约 3% 提升；门与无门差别在噪声内；新层门开率只有 0.16~0.5，说明额外深度用处不大。

### **4.2** 持续学习（**day1 → day2**）

| 组 | 做法 | day1 PPL | day2 PPL |

|---|---|---|---|

| A | 基础（只见过 day1） | 4.64 | 1386.7 |

| B | 全参微调 day2 | 46.3 | 1.52 |

| C | 冻结 + 加 6 层（无门、无回放） | 18.2 | 4.26 |

| D | 冻结 + 加 6 层 + 回放 day1（各 50%） | 4.78 | 8.67 |

| L | 长期学（新层 + 12 层门，day2） | 26.3 | 3.76 |

| LR | 长期 + 回放 | 4.83 | 6.60 |

观察：全参微调 day2 最好但 day1 崩；加层遗忘更轻；回放让 day1 基本保住、day2 略差。

### **4.3** 门作为切换器
```

在 L（长期学完）基础上冻结所有层，只训门：

| 组 | 做法 | day1 PPL | day2 PPL |

|---|---|---|---|

| REC | 只训门 + 回放 day1 | 4.72 | 1415.6 |

| D2 | 只训门 + 回放 day2 | 24.11 | 3.79 |

| DUAL | 只训门 + 双模式回放 | 6.96 | 4.50 |

| DUAL6 | 门只在后 6 层的双模式回放 | 6.11 | 4.48 |

观察：单模式切换零和；双模式回放能让门同时保住两边；门放后 6 层与 12 层差别很小。

4.4 极小回放 **vs** 极小约束

| 组 | 做法 | day1 PPL | day2 PPL |

|---|---|---|---|

| C | 不回放 | 18.2 | 4.26 |

| D | 全量回放（50%） | 4.78 | 8.67 |

| E2 | 100 条旧样本、混合 10% | 9.05 | 5.02 |

| E3 | 100 条旧样本 KL 约束（权重 1.0） | 4.70 | 29.74 |

观察：100 条样本就能显著缓解遗忘；极小回放和全量回放各有优劣（旧能力 vs 新能力）；KL 约束权重 1.0 太强。

4.5 蒸馏压缩

教师 = 12 层 gated（基础 + 6 新层，双模式回放巩固）：

| 模型 | day1 PPL | day2 PPL | 参数量 |

|---|---|---|---|

| 教师（12 层 gated） | 6.96 | 4.50 | ~2.5M |

| 学生 A（8 层，d=160，无门） | 7.42 | 2.18 | 2.68M |

| 学生 B（12 层，d=128，无门） | 7.44 | 2.83 | 2.55M |

观察：学生保留两边能力（day1 差约 6%、day2 更好），门被完全去掉。

五、两个方向

5.1 个人 **AI** 的睡眠巩固

```

· 每人一个"个人层"（新加层 + 门），共享底座

· 白天：上下文/记忆保存（短期、无损）

· 晚上：把当天语境训进个人层，只训个人层和门

· 少量回放共享数据防遗忘

· 定期蒸馏压缩个人层

· 好处：隔离、隐私（数据只进个人层）、可回滚

· 可行性：我们的加层+回放+蒸馏实验支持机制可行；

真实规模未验证

```

5.2 大模型新语料的增量更新

```

· 旧模型冻结，只做推理

· 新语料到达 → 在其上训新层

· 可连续更新，不必全量重训

· 层积累后定期蒸馏打包

· 这是"全量重训"的增量替代，不是免费午餐

· 适用边界：
```

· 领域扩展、同类型新语料：可行

· 需要旧模型从未具备的低层特征（全新语言/模态）：不够，

可能要改嵌入或重训底层

· 代价：前向变贵（多出的层）、蒸馏一次成本

```

两个方向都只到"可行"级别；我们的小实验只验证机制，不验证规模与真实收益。

## 六、与 **Google sleep framework** 的对照

Google 2026 年的睡眠框架（arXiv:2606.03979，*Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories*）把运行分成清醒与睡眠两阶段：

- **Memory Consolidation**："Knowledge Seeding"——把较小自身的记忆向上蒸馏进更大的网络，在扩容同时保住知识；

- **Dreaming**：用 RL 生成合成数据课程，排练新知识、精炼旧能力。

| | Google sleep | 本篇 |

|---|---|---|

| 扩容方式 | 蒸馏进更大网络 | 冻结旧模型 + 加 6 层 |

| 防遗忘 | 向上蒸馏 + 回放 | 双模式回放（教门两种模式） |

| 巩固信号 | RL 生成合成数据 | day1/day2 混合数据 |

| 切换机制 | 无显式门 | 门（单模式零和） |

方向一致：都把"短期记忆 → 长期参数"当核心，都靠回放防遗忘。差异在机制。

## 七、方向判断与局限

**判断（只在玩具规模上成立）**

```

· 加层可以承载新能力，遗忘比全参微调轻

· 门是"细调旋钮"与"切换器"，不是能力开关

· 回放是关键；极小回放（100 条）就有明显效果

· 蒸馏可以把生长打包成固定大小、无门的模型

· "生长 → 巩固 → 打包"构成一个自洽循环

```

**局限**

```

· 极小模型（1.21M）、字符级、单种子

· day1/day2 分布差异极大（莎士比亚 vs 代码）

· 学生用硬标签混合数据训练，未用教师软标签或 RL 课程

· 没有在真实规模模型上验证

· "门放后 6 层略好"的差异在噪声内，不能当结论

```

**结论**

```

· 加层 + 回放 + 蒸馏是一条可行的持续学习循环

· 它的两个可能应用方向：个人 AI 的巩固、大模型的增量更新

· 两者都只到"可行"级别；真实价值需要真实规模验证

```

## 附：标注

- **[事实]** 第四、五节全部 PPL 数字（本地运行结果 JSON）。

- **[方向判断]** 第五节两个方向、第七节判断。
```

- **[不声称]** 不声称方法新颖；不声称可扩展到真实大模型；不声称优于全参微调/外部记忆。

- 相关：Google sleep framework [arXiv:2606.03979](https://arxiv.org/abs/2606.03979)；

[Google Research 博客](https://research.google/blog/the-sleep-framework-a-new-way-for-llms-to-retain-knowledge-in-continual-learning/)。

- 脚本：`continual_add.py`、`continual_time.py`、`continual_gate_dual.py`、`continual_gate_dual6.py`、`small_replay.py`、`distill_compress.py`、`gated_extend.py`、`gated_extend_nogate.py`；结果 JSON 同目录。